SDN IPS Team Installation Runbook

Clean setup steps for new team members: Ubuntu dependencies, Snort 3, Ryu, Open vSwitch, Mininet mirroring, startup order, tests, and troubleshooting.

Item	Lab Default	Change When
Controller/Ryu/Snort VM	192.168.1.200	Teammate uses a different bridged LAN
Mininet VM	192.168.1.201	Teammate uses a different bridged LAN
External test machine	192.168.1.202	Only used for external attack tests
Controller interface	ens33	Check with ip -br addr
Snort bridge	br-snort	Keep this name unless code/docs are updated
VXLAN keys	s1 key 100, ap1 key 101	Must match on both VMs
Repo folder	~/Desktop/GP	Use the real cloned path

Critical rule
Do not install Snort with sudo apt install snort for this project. That usually installs Snort 2. This project uses Snort 3 / Snort++ with Lua config files.

1. Architecture And What The Repo Provides

The project uses two Ubuntu VMs. The controller VM runs Ryu, Snort 3, the alert reader, the Snort-to-Ryu bridge, Open vSwitch, and the br-snort monitoring bridge. The Mininet VM runs the SDN topology and mirrors traffic to the controller VM through VXLAN.

- The repo provides project code, Snort Lua config, local rules, scripts, and documentation.
- The repo does not install system programs by itself. Snort 3, Ryu, OVS, Mininet-WiFi, Python packages, and hping3 must be installed on the Ubuntu machines.
- If IPs or interface names differ, update the commands before running them.
- Old/basic-test Snort files should be commented or disabled, not deleted.

2. Clone The Repo And Check The Branch

```
cd ~/Desktop
git clone https://github.com/D3ath7274/GP.git
cd GP
git status
git remote -v
```

3. Controller VM Static IP

Run on the controller/Ryu/Snort VM. First check the interface and gateway:

```
ip -br addr
ip route | grep default
```

Then set the lab default static IP. Replace IFACE and GATEWAY if your output is different.

```
IFACE=ens33
CONTROLLER_IP=192.168.1.200
GATEWAY=192.168.1.1

sudo tee /etc/netplan/01-sdn-controller-static.yaml >/dev/null <<EOF
network:
  version: 2
  renderer: NetworkManager
  ethernets:
    $IFACE:
      dhcp4: false
      addresses: [$CONTROLLER_IP/24]
      routes:
        - to: default
          via: $GATEWAY
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
EOF

sudo netplan apply
ip -br addr
```

4. Controller VM Dependencies

```
sudo apt update
sudo apt install -y git curl build-essential cmake flex bison pkg-config autoconf \
  automake libtool libpcap-dev libpcre3-dev libpcre2-dev libdumbnet-dev \
  zlib1g-dev libluajit-5.1-dev libhwloc-dev openssl libssl-dev liblzma-dev \
  libunwind-dev uuid-dev libhyperscan-dev libsafec-dev python3 python3-pip \
  openvswitch-switch tcpdump iproute2 net-tools hping3

sudo systemctl enable --now openvswitch-switch
pip3 install --user ryu requests flask eventlet
```

5. Install Snort 3 / Snort++

Run on the controller VM. Remove Snort 2 if it exists, then build DAQ and Snort 3 from source.

```
sudo apt remove -y snort || true
hash -r

cd ~/Downloads
git clone https://github.com/snort3/libdaq.git
cd libdaq
./bootstrap
./configure
make -j"$(nproc)"
sudo make install
sudo ldconfig

cd ~/Downloads
git clone https://github.com/snort3/snort3.git
cd snort3
./configure_cmake.sh --prefix=/usr/local
cd build
make -j"$(nproc)"
sudo make install
sudo ldconfig

/usr/local/bin/snort -V
```

Expected result
The version command must print Snort++ 3.x. If it says Snort 2.x, the wrong binary is being used. Use /usr/local/bin/snort explicitly.

6. Install Project Snort Config And Rules

Run from the cloned repo on the controller VM.

```
cd ~/Desktop/GP

sudo mkdir -p /etc/snort/rules /var/log/snort
sudo cp Controller/snort3/sdn_ips.lua /etc/snort/sdn_ips.lua
sudo cp Controller/snort3/sdn_ips_local.rules /etc/snort/rules/sdn_ips_local.rules

ls -l /etc/snort/sdn_ips.lua /etc/snort/rules/sdn_ips_local.rules
```

The active local rules should be:

```
alert icmp any any -> any any (msg:"ICMP Flood"; itype:8; detection_filter:track by_src, count 10, seconds 5;
  sid:1000001; rev:2;)

alert tcp any any -> any any (msg:"TCP Port Scan"; flags:S; detection_filter:track by_src, count 20, seconds 10;
  sid:1000002; rev:2;)

alert tcp any any -> any 22 (msg:"SSH Brute Force"; flags:S; detection_filter:track by_src, count 5, seconds 60;
  sid:1000003; rev:2;)

alert udp any any -> any 53 (msg:"UDP Flood"; detection_filter:track by_src, count 50, seconds 5; sid:1000004;
  rev:2;)

alert tcp any any -> any 8080 (msg:"Ryu REST API TCP Flood"; flags:S; detection_filter:track by_src, count 20,
  seconds 5; sid:1000005; rev:1;)
```

```
sudo /usr/local/bin/snort -T -c /etc/snort/sdn_ips.lua -i ens33
sudo /usr/local/bin/snort -T -c /etc/snort/sdn_ips.lua -i br-snort
```

About the second test
The br-snort test works only after the bridge exists. If br-snort does not exist yet, create the permanent controller bridge in the next section.

7. Make Controller br-snort Permanent

Run on the controller VM. This creates br-snort, VXLAN ports for s1 and ap1, and a tc mirror from the external interface into br-snort.

```
sudo tee /usr/local/sbin/setup-br-snort.sh >/dev/null <<'EOF'
#!/usr/bin/env bash
set -e

MININET_IP="192.168.1.201"
EXT_IF="ens33"

ovs-vsctl --may-exist add-br br-snort

ovs-vsctl --may-exist add-port br-snort vxlan-s1 \
  -- set interface vxlan-s1 type=vxlan \
  options:remote_ip=$MININET_IP options:key=100

ovs-vsctl --may-exist add-port br-snort vxlan-ap1 \
  -- set interface vxlan-ap1 type=vxlan \
  options:remote_ip=$MININET_IP options:key=101

ip link set br-snort up

ip link show veth-mirror0 >/dev/null 2>&1 || \
  ip link add veth-mirror0 type veth peer name veth-mirror1

ip link set veth-mirror0 up
ip link set veth-mirror1 up
ovs-vsctl --may-exist add-port br-snort veth-mirror1

tc qdisc del dev $EXT_IF ingress 2>/dev/null || true
tc qdisc add dev $EXT_IF ingress
tc filter add dev $EXT_IF parent ffff: protocol ip u32 match u32 0 0 \
  action mirred egress mirror dev veth-mirror0

ovs-vsctl show
EOF

sudo chmod +x /usr/local/sbin/setup-br-snort.sh


sudo tee /etc/systemd/system/br-snort.service >/dev/null <<'EOF'
[Unit]
Description=Create br-snort monitoring bridge for SDN IPS
After=network-online.target openvswitch-switch.service
Wants=network-online.target
Requires=openvswitch-switch.service

[Service]
Type=oneshot
ExecStart=/usr/local/sbin/setup-br-snort.sh
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable br-snort.service
sudo systemctl start br-snort.service

sudo systemctl status br-snort.service --no-pager
sudo ovs-vsctl show
ip -br link | grep -E "br-snort|vxlan|veth-mirror"
```

8. Mininet VM Static IP And Dependencies

Run on the Mininet VM. First check interface and gateway.

```
ip -br addr
ip route | grep default
```

```
IFACE=ens33
MININET_IP=192.168.1.201
GATEWAY=192.168.1.1

sudo tee /etc/netplan/01-sdn-mininet-static.yaml >/dev/null <<EOF
network:
  version: 2
  renderer: NetworkManager
  ethernets:
    $IFACE:
      dhcp4: false
      addresses: [$MININET_IP/24]
      routes:
        - to: default
          via: $GATEWAY
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
EOF

sudo netplan apply
ip -br addr


sudo apt update
sudo apt install -y git python3 python3-pip openvswitch-switch tcpdump \
  iproute2 net-tools hping3 iw wireless-tools wpasupplicant hostapd

sudo systemctl enable --now openvswitch-switch
```

Mininet-WiFi

If Mininet-WiFi is not installed on the VM, install it using the team-approved method or the official Mininet-WiFi installer. Verify that sudo mn -c works before running the topology.

9. Make Mininet Mirror Automatic

Run on the Mininet VM. This watcher waits for s1 and ap1 to exist, then applies the VXLAN mirror. It is needed because Mininet recreates OVS bridges when the topology starts.

```
sudo tee /usr/local/sbin/mininet-snort-mirror-watch.sh >/dev/null <<'EOF'
#!/usr/bin/env bash

CONTROLLER_IP="192.168.1.200"

configure_s1() {
  ovs-vsctl --if-exists clear bridge s1 mirrors
  ovs-vsctl --if-exists del-port s1 vxlan-snort-s1
  ip link delete vxlan-snort-s1 2>/dev/null || true

  ovs-vsctl add-port s1 vxlan-snort-s1 \
    -- set interface vxlan-snort-s1 type=vxlan \
    options:remote_ip=$CONTROLLER_IP options:key=100

  ovs-vsctl \
    -- --id=@p get port vxlan-snort-s1 \
    -- --id=@m create mirror name=snort-mirror-s1 select-all=true output-port=@p \
    -- set bridge s1 mirrors=@m
}

configure_ap1() {
  ovs-vsctl --if-exists clear bridge ap1 mirrors
  ovs-vsctl --if-exists del-port ap1 vxlan-snort-ap1
  ip link delete vxlan-snort-ap1 2>/dev/null || true

  ovs-vsctl add-port ap1 vxlan-snort-ap1 \
    -- set interface vxlan-snort-ap1 type=vxlan \
    options:remote_ip=$CONTROLLER_IP options:key=101

  ovs-vsctl \
    -- --id=@p get port vxlan-snort-ap1 \
    -- --id=@m create mirror name=snort-mirror-ap1 select-all=true output-port=@p \
    -- set bridge ap1 mirrors=@m
}

while true; do
  if ovs-vsctl br-exists s1; then configure_s1; fi
  if ovs-vsctl br-exists ap1; then configure_ap1; fi
  sleep 10
done
EOF

sudo chmod +x /usr/local/sbin/mininet-snort-mirror-watch.sh


sudo tee /etc/systemd/system/mininet-snort-mirror.service >/dev/null <<'EOF'
[Unit]
Description=Auto configure Mininet OVS mirrors to Snort controller
After=network-online.target openvswitch-switch.service
Wants=network-online.target
Requires=openvswitch-switch.service

[Service]
Type=simple
ExecStart=/usr/local/sbin/mininet-snort-mirror-watch.sh
Restart=always
RestartSec=3

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable mininet-snort-mirror.service
sudo systemctl start mininet-snort-mirror.service
sudo systemctl status mininet-snort-mirror.service --no-pager
```

10. Start The System

On the controller VM, open four terminals and start them in this order.

```
# Terminal 1 - Ryu controller
cd ~/Desktop/GP/Controller
ryu-manager ryu_ips_app.py --observe-links

# Terminal 2 - Snort-to-Ryu bridge
cd ~/Desktop/GP/Controller
python3 snort_ryu_bridge.py

# Terminal 3 - Snort alert reader
cd ~/Desktop/GP/Controller
sudo python3 snort_alert_reader.py

# Terminal 4 - Snort 3 IDS
sudo mkdir -p /var/log/snort
sudo touch /var/log/snort/alert_json.txt
sudo chmod 666 /var/log/snort/alert_json.txt
sudo /usr/local/bin/snort -c /etc/snort/sdn_ips.lua -i br-snort -l /var/log/snort
```

On the Mininet VM, start the topology and keep the Mininet prompt open.

```
sudo mn -c
cd ~/Desktop/GP/"SDN Topology"
sudo python3 topology.py
```

After topology starts, verify that s1 and ap1 exist and mirrors were created.

```
sudo ovs-vsctl show
sudo ovs-vsctl list mirror
```

11. Traffic Tests

Run these tests in order. Keep Snort, the reader, the bridge, and Ryu visible while testing.

Test	Where	Command	Expected Result
Basic reachability	Mininet prompt	pingall	Hosts and stations can reach each other before blocking
Specific ping	Mininet prompt	h1 ping -c 3 h2	Traffic appears on br-snort tcpdump
ICMP flood	Mininet prompt	h1 ping -f h2	SID 1000001 alert, reader blocks source through Ryu
TCP scan/flood	Mininet prompt	h1 hping3 -S -p 80 -c 30 h2	SID 1000002 alert after threshold
SSH brute force	Mininet prompt	h1 hping3 -S -p 22 -c 10 h2	SID 1000003 alert
UDP flood	Mininet prompt	h1 hping3 --udp -p 53 -c 80 h2	SID 1000004 alert
Ryu API flood	External machine	hping3 -S -p 8080 -c 30 192.168.1.200	SID 1000005 alert, external source blocked by iptables

Useful verification commands:

```
# Controller VM
sudo tcpdump -i br-snort -nn -c 20
sudo tail -f /var/log/snort/alert_json.txt
curl -s http://127.0.0.1:8080/ips/status
curl -s http://127.0.0.1:8080/ips/blocked | python3 -m json.tool
sudo iptables -S INPUT | grep 192.168.1.202

# Mininet VM
sudo ovs-ofctl dump-flows s1 -O OpenFlow13 | grep drop
sudo ovs-ofctl dump-flows ap1 -O OpenFlow13 | grep drop
```

12. Troubleshooting

Symptom	Likely Cause	Fix
Unknown alert option alert_json	Using old Snort command style or Snort 2	Use Snort 3 and run without -A alert_json because alert_json is configured in sdn_ips.lua
Lua config cannot load	/etc/snort/sdn_ips.lua not installed or Snort 2 is running	Copy Controller/snort3/sdn_ips.lua to /etc/snort and verify /usr/local/bin/snort -V
br-snort: No such device	Controller OVS bridge not created	Start br-snort.service or run /usr/local/sbin/setup-br-snort.sh
ovs-vsctl command not found	Open vSwitch not installed	sudo apt install -y openvswitch-switch && sudo systemctl enable --now openvswitch-switch
VXLAN File exists	Stale VXLAN interface from previous topology	Delete stale interface or use unique names vxlan-snort-s1 and vxlan-snort-ap1
Snort sees traffic but no local alert	Wrong rule file or threshold too high	Check /etc/snort/sdn_ips.lua includes sdn_ips_local.rules and expect text rules: 5
External attack alerts but not blocked	Reader not running as sudo or iptables logic not triggered	Run sudo python3 snort_alert_reader.py and check sudo iptables -S INPUT
Internal alert appears but no DROP flow	Bridge or Ryu REST path failed	Check snort_ryu_bridge.py, Ryu /ips/status, and ovs-ofctl drop flows

13. Final Checklist

- Controller VM has static IP 192.168.1.200 or the docs/config are updated to the real IP.
- Mininet VM has static IP 192.168.1.201 or the VXLAN commands use the real IP.
- /usr/local/bin/snort -V prints Snort++ 3.x.
- sudo /usr/local/bin/snort -T -c /etc/snort/sdn_ips.lua -i br-snort passes.
- Snort test output shows Loading /etc/snort/rules/sdn_ips_local.rules and text rules: 5.
- br-snort.service is enabled on the controller VM.
- mininet-snort-mirror.service is enabled on the Mininet VM.
- Ryu sees s1 and ap1 connected.
- tcpdump on br-snort sees internal Mininet traffic and external controller-facing traffic.
- Snort reader shows BLOCKING alerts and Ryu/iptables installs the block.